
How can bees colour graphs

Malika Bessedik*

Laboratoire des Méthodes de Conception de Systèmes (LMCS),
E.S.I ex I.N.I., BP 68M Oued Smar 16309, Alger, Algérie
Fax: +213-21-51-61-56
E-mail: m_bensedik@esi.dz

*Corresponding author

Bouakline Toufik

Institut National d'Informatique,
E.S.I ex I.N.I., BP 68M Oued Smar 16309, Alger, Algérie
Fax: +213-21-51-61-56
E-mail: tbouakline@gmail.com

Habiba Drias

Laboratoire de Recherche En Intelligence Artificielle,
Dept. d'Informatique, USTHB University, Alger, Algérie
E-mail: h_drias@hotmail.fr

Abstract: Marriage in honey bees optimisation (MBO) is a recent evolutionary metaheuristic inspired by the bees reproduction process. Contrary to most of swarm intelligence algorithms such as ant colony optimisation (ACO), MBO uses self-organisation to mix different heuristics. In this paper, we present an MBO approach for the graph colouring problem (GCP). We propose, as worker, in our algorithm (BeesCol) one of the following methods: local search, taboo search or a proposed-based ant colony system algorithm (IACSCol). The worker intervenes at two levels; it improves initial and crossed solutions. Moreover, in BeesCol, one or several queens are generated randomly or by a specific constructive method, namely, recursive largest first or DSATUR. Experimental results on some well studied Dimacs graphs are reported. A comparison between BeesCol and some best-known algorithms for the GCP (hybrid colouring algorithm HCA, ant system and ant colony system) shows that the use of taboo search as worker in BeesCol reached most of best known results.

Keywords: graph colouring problem; GCP; metaheuristics; marriage in honey bees optimisation ant colony system; Tabu search; TS; hybrid colouring algorithm; HCA; DSATUR; RLF.

Reference to this paper should be made as follows: Bessedik, M., Toufik, B. and Drias, H. (2011) 'How can bees colour graphs', *Int. J. Bio-Inspired Computation*, Vol. 3, No. 1, pp.67–76.

Biographical notes: Malika Bessedik obtained her degree in Mathematics, then her Master's degree in Mathematics, specialty in Operational Research, from the Faculty of Mathematics, USTHB University, Algiers, Algeria. She is a Lecturer at the National Institute of Computer Sciences, INI, Algiers, Algeria since September, 1998. Currently, she is teaching the operations research course and preparing a Doctorate in Computer Sciences, specialty in Operational Research at INI. She is a Researcher (optimisation and evolutionary algorithms team) in the laboratory, Laboratoire des Méthodes de Conception de Systèmes (LMCS), E.S.I ex INI.

Bouakline Toufik obtained his Master's degree in Computer Sciences from the National Institute on Computer Sciences, E.S.I ex INI, Algiers, Algeria. He was a member of the Optimisation Research Team during the year 2006 in the LMCS, E.S.I ex INI. He is currently working as an Engineer in GENI SOFT INFORMATIQUE, Algiers, Algeria.

Habiba Drias holds his Master degree in Computer Science from Case Western Reserve University, Cleveland, Ohio, USA (1984) and his Doctorate degree prepared at Paris University from Algiers USTHB University (1993). She has directed L'école Nationale Supérieure d'informatique – E.S.I ex I.N.I for many years. She is currently a Lecturer in the Computer Sciences Departement of USTHB University and directs the Laboratory of Research in Artificial Intelligence, LRIA. She has over 100 published papers in the domain of artificial intelligence, e-commerce, computational complexity and the satisfiability problem.

1 Introduction

A colouring of a graph G is an affectation of a colour to each node of G such that any two adjacent nodes have different colours. The graph colouring problem (GCP) is an optimisation problem that consists of finding a colouring for a given graph G that uses a minimum number of colours. GCP is one of the most studied NP-hard problems in graph's theory, completeness theory and operational research. Its importance is justified by its diverse and interesting applications such as scheduling (Marx, 2004), register allocation (Smith et al., 2004), and frequency assignment (Zufferey, 2002).

Swarm intelligence is based on the collective behaviour of decentralised, self organised systems. More and more researchers are interested in this new exciting way of achieving a form of artificial intelligence of simple agent groups. Modelling the behaviour of social insects, such as ants and bees, and using these models for search and problem solving are the context of the emerging area of swarm intelligence. Ants and bees are among the well-studied social insects. Ant colony is a typical successful swarm-based approach to optimisation (Bonabeau et al., 1999). The number of their successful applications is exponentially growing in combinatorial optimisation, communication networks, and robotic, (Gambardella and Dorigo, 2000; Walkowiak, 2005; Murtadha Mohamad, 2008). Moreover, honey bees exhibit a model for intelligent behaviour used in real word engineering optimisation problems. Marriage in honey bees optimisation (MBO) is a recent evolutionary metaheuristic inspired by the reproduction process in honeybees (Abbass, 2001).

In this paper, we first present the components of an MBO approach for the GCP as well as the improvement methods used in this approach namely; Taboo search (TS), local search (LS) and a proposed-based ant colony system algorithm (IACSCol). Then, we carry out a comparative study between our algorithm and some best-known algorithms for the GCP, namely: ant system, a constructive ant colony system algorithm, and a hybrid colouring algorithm (HCA).

In Section 2, we present the GCP and a brief review of different approaches to solve it. An outline of the MBO general concept is given in Section 3. In Section 4, we introduce our algorithm called BeesCol. In Section 5, we present the methods used in BeesCol to improve each initial and crossed solution. In Section 6, we discuss the tests results and compare all implemented methods, and finally, in Section 7, we draw some conclusions and give an outlook of future works.

2 The GCP

Given an undirected graph $G = (X, V)$, a colouring of G consists of assigning to each node of X a colour different from those of its neighbours (adjacent nodes). Two adjacent nodes are said to be in conflict if they have the same colour.

Formally, a colouring of G is a map $c: X \rightarrow N$ such that $c(x) \neq c(y)$ whenever $(x, y) \in V$. If the colouring uses k colours then it is called a k -colouring. It can be stated as a vertex partitioning problem into k stables called colour classes. Each stable contains the vertices with the same colour. The minimal number of colours for which a colouring exists is called the chromatic number of G and is denoted by $\chi(G)$. An optimal colouring is one that uses exactly $\chi(G)$ colours. The GCP is the optimisation problem that consists of finding an optimal colouring for a given graph G .

The GCP is computationally hard. It is NP-complete to decide if a given graph admits a k -colouring for a given k except for the cases $k = 1$ and $k = 2$. In fact, it is NP-hard to compute the chromatic number. GCP remains NP-complete even on planar graphs of degree 4. "One of the best current algorithms for the GCP is a hybrid algorithm named HCA" (Galinier and Hao, 1999). "It combines a genetic algorithm with Tabu search. The Success of HCA is a direct consequence of its original crossover operator" (Glass and Bennett, 2003).

There are many resolutions approaches for the GCP. Greedy constructive methods such as, DSATUR (Brelaz, 1979), RLF (Leighton, 1979) are often used to generate an initial solution and remain a comparative reference for evaluating new methods. LS methods, such as TS (Hertz and de Werra, 1987) are generally used in hybrid methods. Nowadays, most used methods are evolutionary ones, such as scatter search (Hamiez and Hao, 2002), ant system (Costa and Hertz, 1997) or ant colony system (Bessedik et al., 2005; Dorigo et al., 1999; Stulzle and Dorigo, 1999). Nevertheless most efficient algorithms are the hybrid ones (Dorne and Hao, 1998) and (Glass and Bennett, 2003; Fleurent and Ferland, 1996). The best hybrid approach combines an evolutionary algorithm with an LS (Glover et al., 1995). However, their main drawbacks are their high running time.

3 Marriage in honey bees optimisation

Bees are social insects living in very organised colonies. Each honey-bees colony consists of one or several queens, drones, workers and broods. Queens specialise in egg laying, workers in brood care and sometimes egg lying, drones are the males of the colony and broods the children. MBO is a new swarm approach to combinatorial optimisation problems inspired by the bee's reproduction process. Contrary to most swarm intelligence algorithms such as ant colony optimisation, MBO uses self-organisation to mix different heuristics (Abbass, 2001a). MBO was initially conceived for the 3-SAT problem (Abbass, 2002), and used to solve other problems such as Max-W-Sat (Benatchba et al., 2005) and partitioning (Koudil et al., 2007).

The reproduction process of real bees can be summarised as follows: during a mating flight, each queen mates with seven to 20 drones. During each mating, sperm reaches the queen spermatheca and accumulates there to

form the genetic pool of the colony. Each time a queen lays eggs (broods), it retrieves at random a mixture of sperm from its spermatheca to fertilise an egg. The next paragraph gives details of the simulation of this process in MBO (Abbass, 2001b).

The algorithm starts with initialising the queen's genotype (a complete solution) at random. The queen is then improved by the worker (heuristic). A number of mating flight is undertaken. In each mating flight, the queen's speed and energy are randomly initialised. During this flight, and according to its speed, the queen collects sperm of different drones (a complete solution) with a certain probability. In other words, if the drone is successfully mated with the queen, its sperm is added to the queen's spermatheca. When the queen's energy reaches a critical threshold or when its spermatheca is full, it returns to the nest and the brood creation process starts. Several broods are generated by crossing over the queen genome with a sperm selected at random in its spermatheca. A sperm is a partial solution which is completed by the queen's genome. The resulting broods are improved by the worker. If the best brood is better than the queen, then it replaces it. The remaining broods are destroyed, and a new nuptial flight starts. This process is repeated until all the nuptial flights are undertaken or a stopping criterion is reached.

4 How can bees colour graphs

Before giving the framework and the components of our MBO approach of for the GCP, we present the graph and

the solutions encoding as well as the objective function used.

4.1 Encoding and objective function

As stated before, the goal of the GCP is to find a valid colouration (a feasible solution) which minimises the number of colours used to colour a graph $G = (X, V)$. A graph $G = (X, V)$ is represented by its adjacency matrix (node, node) $M = (M_{ij})$ where: $M_{ij} = 1$ if x_i and x_j are adjacent, and 0 if not.

As we use the partition approach of the GCP, a solution (colouration) is represented by a set of colour classes. Formally, the objective function is defined as follows: Let $S = \{V_1, V_2, V_3, \dots, V_q\}$ be a solution to evaluate such as:

$$\bigcup_{i=1}^q V_i = V \text{ and } V_i \cap V_j = \emptyset; V_i, V_j \subset V$$

for $1 \leq i \leq q$ and $1 \leq j \leq q$

The objective function f to minimise is therefore: $|S| = q$, where q is the number of colour classes; it is also called the partition number.

4.2 BeesCol algorithm

In our algorithm which we called BeesCol, several queens are used contrary to (Abbass, 2001) who used a single one. In BeesCol, one can choose a worker among several implemented methods: LS, TS or a based ant colony system algorithm (IACS). The steps of BeesCol are given below:

Figure 1 BeesCol algorithm

```

Begin
  Initialise number of queens (nb_Q), number of mating flight (nb_mf), spermatheca size (Sper)
  Initialise worker with one of these metaheuristics: TS, LS or IACSCol
  Generate nb_Q queens randomly or using a constructive algorithm DSATUR or RLF (1)
  Improve the genotype of each queen by the chosen worker (2)
  For each queen Q do
    Initialise the speed and energy for the queen (3)
    Step = 0,5 * energy/sper
    While nb_mf is not reached do
      Generate a drone (4)
      While energy of queen > 0 do
        If the queen's spermatheca is not full,
          If the drone is selected according to the probability rule (2)
            Add the sperm of the selected drone to the spermatheca (5)
          EndIf
        EndIf
      Update energy and speed according to formulas (3 and 4) (6)
      Change a colour in the drone genotype (7)
    Endwhile
    For each sperm in the spermatheca do
      Generate a brood by crossing over the queen's genotype and the selected sperm (8)
    EndFor
    Improve each brood using the worker (TS or LS), or use all broods as an initial population which is improved by IACSCol if
    this metaheuristic is chosen as a worker (9)
    Replace the current queen with the best brood found if it improves the queen's genotype (10)
  Endwhile
Endfor
End

```

At the beginning of the algorithm, we initialise some parameters: the number of queens (nb_Q), the number of nuptial flights (nb_mf) for each queen, the coupling probability between a queen and a drone Q_0 , the spermatheca size which represents a maximum number of sperms that the queen can accumulate, the parameter of the update rule of the queen speed $\alpha \in]0,1[$ and finally the worker TS, LS or IACSCol.

- 1 Several queens (initial solutions) are generated either by assigning a colour randomly to each node of the graph, or using the constructive algorithm RLF or DSATUR (Costa, 1995).
- 2 Initial solutions (queens) are improved by the worker. If TS or LS is chosen, then each queen (initial solution) is improved separately by this worker. If IACS is chosen to be a worker, then all queens are considered as an initial population used by IACSCol to be improved.
- 3 For each improved queen Q, speed and energy are initialised randomly in $[0.5, 1]$ making sure that Q will fly during a certain time. The transitions (i.e., the moves from a solution to another) carried out by each queen are done according to its speed and energy.

At each stage of the algorithm, the queen consumes an amount of energy called step. It decreases each time a mating occurs. Step is computed according to the formula:

$$\text{Step} = \frac{0.5 * E0}{\text{Spermatheca size}} \quad (1)$$

While the number of matting flights is not reached, the following steps are executed:

- 4 Generate a drone D (a complete solution eventually invalid): for each drone genotype, a mask is used to hide half of its genes. The generation of the mask is done randomly.
- 5 To evaluate the ability of the drone D to be added to the queen's spermatheca (if not full), we compute the probability $\text{Prob}(Q, D)$:

$$\text{prob}(Q, D) = e^{(-\text{dif}/s(t))} \quad (2)$$

where dif is the absolute difference between the queen Q and the drone D costs divided by their sum, $S(t)$ is the queen speed at the iteration t . One can notice, according to this formula that, if the drone's cost (number of colours) is close to the queen's cost and the queen's speed is high, then the drone is more likely to be selected.

- 6 After each mating (t), the energy and speed of the queen is updated according to these formulas:

$$S(t+1) = \alpha * S(t). \quad (3)$$

Such as $\alpha \in]0,1[$

$$\text{Energy}(t+1) = \text{Energy}(t) - \text{step} \quad (4)$$

- 7 To simulate the move of the queen towards another solution, at the next iteration of the nuptial flight, one changes a node colour of the drone genotype; this corresponds to the move of the chosen node from its partition to another partition.

As long as the queen energy is positive, it carries out stages 1–7.

- 8 Each queen carries out a crossover between its genotype and all sperms present in its spermatheca using the following steps:

For each drone, we generate randomly a sperm. The drone's sperm misses half of its genes (masked genes). Recall that for the GCP, a gene in a drone genotype represents a node colour. Consequently, the broods are obtained by completing missing genes in the drone sperm by the corresponding queen's genes.

- 9 The broods resulting from the crossover of each queen with the drones are improved using one of these two strategies:
 - each brood is improved by TS or LS if one of these two metaheuristics is chosen as worker
 - all broods are used as an initial population which is improved by IACSCol if this metaheuristic is chosen as worker.
- 10 The queen is replaced by the best brood found at Stage 9 if it is better than the current queen.

MBO algorithm finishes when each queen carries out a number of matting flights.

5 Improvement methods (workers)

In BeesCol, we use as worker one of these three methods: an Improving ant colony system for the GCP (IACSCol), TS, or LS to improve each initial solution as well as each solution resulting from the crossover. We present below these methods focusing on IACSCol as we conceived it.

5.1 The algorithm IACSCol

In IACSCol, the search space is a graph where every node is a complete solution. Each node is connected to all its neighbours (two neighbouring solutions differ by a single colour). Each ant has initially a complete solution. It is either an initial queen if IACSCol intervenes to improve the initial solutions or a brood if it intervenes to improve the crossed solutions. In IACSCol, an ant moves, at each iteration, to another neighbour. When moving, the ant changes a node colour (recolours) by displacing it from a stable S_i to another stable S_j . We give below the steps of IACSCol algorithm:

Figure 2 IACSCol algorithm

Begin
Initialisation of the pheromone trails and other parameters (1)
While not stopping criteria do
 For each ant A_i do
 Assign S_i (queen or brood) to ant A_i (2)
 Recolour a node (3)
 Apply online delayed update of pheromone (4)
 End For
 Determine the best solution found in this iteration (5)
 Apply offline update of pheromone (6)
 Apply evaporation in case of stagnation (7)
End While
End

- 1 Initialisation of pheromone trails to a value τ_0 and the following parameters: The colony size, the number of generations (stopping criterion), the transition rule parameters α , β , q_0 and pheromone trails persistence $\rho \in [0, 1]$.
- 2 Assign the solution S_i to ant A_i : S_i is either an initial queen if IACSCol is called to improve the initial solutions or a brood if it is called to improve the crossed solutions.
- 3 Recolour a node: Is a procedure (Figure 3) which affects a new colour to this node. This corresponds to move the chosen node to another stable in order to minimise the conflicts if they exist or to improve the solution. The recoloured node is added to the ant tabu list. This process is repeated a number of iterations fixed a priori.

Figure 3 IACSCol algorithm

Algorithm recolour a node

Begin
While not stopping criteria do
 Generate a random number $q \in [0, 1]$
 If $q \leq q_0$ then
 Select the solution s whose product between the pheromone and the heuristic information values is maximum, according to formula (5)
 Else
 S is chosen by the probability (6)
 EndIf
 If s exists in the current solution being built by the ant then
 Recolour s (move it to another partition)
 Else
 Add s in the current solution
 EndIf
EndWhile
End

Where formulas (5) and (6) are given below:

$$s = \arg \max \left\{ \left(\tau_{ij} \right)^\alpha (t) \cdot \left(\eta_{ij} \right)^\beta (t) \right\} \quad (5)$$

$$\left(\tau_{ij}(t) \right)^\alpha \cdot \left(\eta_{ij}(t) \right)^\beta \quad (6)$$

Such that $\tau_{ij}(t)$ and $\eta_{ij}(t)$ are respectively, the pheromone value on the edge (i, j) and the heuristic information on the node j , in iteration ' t ' (we give more details on the heuristic information thereafter). q_0 ($0 \leq q_0 \leq 1$) is a parameter of the algorithm which determines the relative importance between the exploitation and the exploration. α and β are two parameters which control the relative influence of the pheromone trails and heuristic information and J_i^k is the feasible neighbourhood of the ant k ; that is, the set of nodes which the ant has not yet visited.

The choice of the next node to colour is influenced by the heuristic information related in the case of the GCP to the characteristic of the nodes (degree, degsat...). We propose two types of heuristic information: static and dynamic. The first one represents the degree of the node to colour. While the second one, it is defined as the saturation degree of the node to colour. Let's recall that the saturation degree of a node v (DSATUR(v)) is the number of colours already assigned to the neighbours of v . In this case, the heuristic information values are recomputed at each iteration.

- 4 Online delayed update: Once the solution is improved, the ant updates the pheromone trail. The amount of added pheromone is proportional to the cost of the found solution. We have implemented two strategies of pheromone deposit: pheromone on nodes or on connections. The online update of pheromone is given according to the following formula:

$\tau_{ij} = \tau_{ij} + \rho / \text{cost}(S^*)$ ((pheromone on nodes) or $\tau(s) = \tau(s) + \rho / \text{cost}(S^*)$ (pheromone on connections), where S^* represents the best solution found until now, cost is the number of conflicts in the solution and $\rho \in [0, 1]$ is the pheromone trails persistence.

- 5 At the end of the activities of one ants' generation, we determine the best solution found by this generation and we update the best global solution if it has been improved.
- 6 Offline update: Extra-pheromone is added to the best solution found in this generation to intensify the search around this solution. The offline update is carried out according to the following formula:

$$-\forall s \in S^*, t(s) = (1 - \mu) * \tau(s) + \mu * \text{cost}(S^*) \quad (7)$$

(case of pheromone on nodes).

$$-\tau_{ij} = (1 - \mu) * \tau_{ij} + \mu * \text{cost}(S^*) \quad (8)$$

(case of pheromone on connexions)

where S^* is the best solution found until now, and μ is a parameter indicating the pheromone decrease and

$\text{cost}(S)$ is defined as the number of colours used in the solution S .

- 7 Evaporation: If the same solution appears in several successive iterations of the algorithm (stagnation), then an evaporation of the pheromone is applied according to this equation:

$$\tau(v_i, v_j) = (1 - \rho) * \tau(v_i, v_j) \quad (9)$$

The algorithm stops when a number of iterations or a maximum execution time defined a priori is reached.

5.2 Two other workers

Beside IACSCol, BeesCol uses two LS methods as workers. TS is an intelligent metaheuristic which has several mechanisms to explore efficiently the search space. It allows focusing the search within an interesting area as well as leading the search to non-explored areas. Furthermore, TS has proven its effectiveness for the GCP. LS is used, in this work, for its simplicity and good average-case behaviour. Moreover, LS combined with an evolutionary algorithm gives good results.

5.2.1 TS for the GCP

We use TS to improve locally each initial queen or each brood. TS used in this work is Tabucol proposed by (Hertz and de Werra, 1987). Even though there exists in the literature more performing LS algorithms, Tabucol is still often used in hybrid algorithms that combine a LS with an evolutionary algorithms (Galinier and Hertz, 2006). Tabucol principle is summarised as follows: From an initial solution (an initial queen or a brood), which is not necessarily valid, some nodes are recoloured to obtain a valid colouring using a minimum of colours. A neighbour of a solution (colouring) is obtained by moving a node v from a stable S_i to another stable S_j . The node v to be moved to S_j is chosen randomly among the nodes in conflict with most of vertices in its partition S_i . This process is repeated a certain number of times to obtain different solutions. The selection of a solution in the neighbourhood is done according to one of these strategies: best move, first improves or randomly. A couple (v, c) where c is the colour assigned to the node v is taboo for the 'it' next iterations. In other words, v cannot receive the colour c during the next 'it' iterations. 'It' depends on the number of nodes in conflict in the current solution. In addition, we have used the aspiration criterion so that a taboo move which leads to a solution better than the best global one is always accepted.

5.2.2 LS for the GCP

LS is an iterative method that explores the search area from an initial solution improving at each iteration the current one. We have implemented a basic version of LS (Galinier and Hertz, 2006). The principle of LS for the GCP in this work is summarised as follows: initial solution is a queen or a brood. Since the initial queen can be generated randomly,

the current solution can be invalid. At each iteration, the algorithm improves the current solution. Therefore, the aim of LS in our algorithm is to obtain a solution with fewer colours or conflicts. A new solution s is obtained by changing the least recently modified node colour which improves the current solution s_c . If the cost of s is better than the one of s_c , then s_c is replaced by s . Otherwise, LS seeks a solution with the same cost as s_c and the search starts again from this new solution. The research process stops if the solution cost stagnates during 'Smax' iterations.

6 Tests results

A random graph G_n, p , is a graph with n vertices and such that there exists an edge with a probability p between any pair of vertices. Random graphs are deeply studied with respect to colouring, especially for $p \in \{0.4, 0.5, 0.6\}$. There are many generators for this class of graphs. In this work, we use the same generator as in Costa and Hertz (1997).

In this section, we first present a summary of tests performed to adjust the different parameters of our algorithm and study the influence of each worker on the behaviour of BeesCol. Then, we do a comparative study between BeesCol and some best known ant colouring algorithms, namely, ant system (Costa and Hertz, 1997) and ant colony system (Bessedik et al., 2005). These tests are done on the random graphs G_n, p , with $p \in \{0.4, 0.5, 0.6\}$.

Finally, we compare BeesCol to HCA (Galinier and Hao, 1999) on four benchmarks used by Galinier and Hao from the well-known second DIMACS. Finally, we conclude with a synthesis.

We use the following notations in this section: improving ant colony system (IACSCol), TS and LS, HCA. In BeesCol, we use these abbreviations for the queen's parameters.

Sper	spermatheca size
nb_mf	number of mating flights
S_0	initial speed
E_0	initial energy
α	rate of speed decline
Q_0	probability of successful mating μ .

We have also, nb_Q is the queens number and work the worker used in BeesCol (LS, RT or ACS2).

6.1 Adjustment of BeesCol parameters

We have performed several tests to select appropriate parameters values for BeesCol.

These experiments are carried out on six random benchmarks G_n, p among the most difficult with $n \in \{100, 300\}$ and $p \in \{0.4, 0.5, 0.6\}$. In what follows, we denote by n_β , the random graph G_n, p whose number of nodes is n and density $0.\beta$ ($p = 0.\beta$). We have considered

various parameters and several executions for each method and each benchmark.

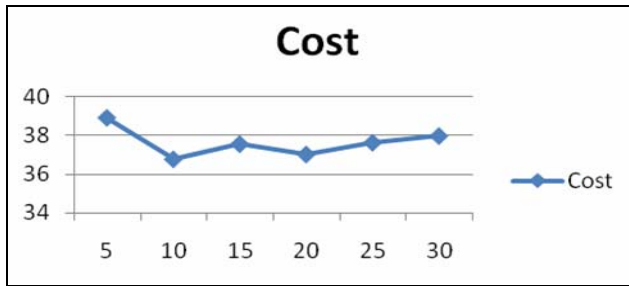
Table 1 is a summary of all performed tests. The results of each line (benchmark) are the average of 20 executions. A close look to Table 1 shows that the best parameters for almost all runs are: $nb_Q = 10$, $Sper = 9$; $nb_mf = 10$; $S_0 = 0.7$; $E_0 = 0.6$; $\alpha = 0.5$ and $Q_0 = 0.8$.

Table 1 Adjustment of BeesCol parameters

Graph	nb_Q	$Sper$	nb_mf	S_0	E_0	α	Q_0	Work	Cost
100_4	5	2	5	0.9	0.6	0.4	0.8	LS	14.62
100_4	10	9	10	0.9	0.6	0.3	0.8	LS	13.3
100_4	7	3	7	0.7	0.6	0.9	0.4	IACScol	14.34
100_5	10	10	20	0.7	0.6	0.5	0.8	TS	16.6
100_5	20	10	10	0.5	0.5	0.6	0.7	TS	16.83
100_5	20	10	10	0.5	0.5	0.6	0.7	LS	16.03
100_6	10	9	10	0.7	0.6	0.3	0.8	TS	18.59
100_6	5	3	5	0.9	0.6	0.6	0.8	IACScol	20.12
100_6	10	9	10	0.7	0.6	0.3	0.8	IACScol	19.98
300_4	10	9	10	0.7	0.6	0.4	0.8	TS	28.87
300_4	5	2	5	0.9	0.7	0.6	0.9	TS	30.32
300_4	10	9	10	0.7	0.6	0.4	0.8	LS	31.02
300_5	10	20	20	0.7	0.6	0.3	0.8	LS	39.59
300_5	5	4	6	0.9	0.6	0.6	0.8	IACScol	37.92
300_5	10	9	10	0.7	0.6	0.4	0.8	TS	35.91

Moreover, we have performed tests on the benchmark 300-5 to show the influence of the queens' number on the cost (Figure 4).

Figure 4 Influence of nb_Q on the cost (see online version for colours)

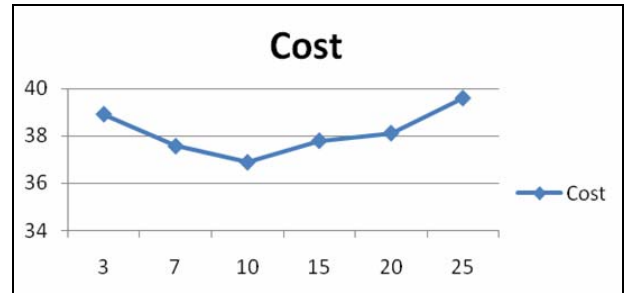


Notes: $Sper = 9$; $nb_mf = 10$; $S_0 = 0.7$; $E_0 = 0.6$; $\alpha = 0.5$ and $Q_0 = 0.8$, work = TS)

One can notice that when using a low number of queens ($nb_Q < 9$), the cost of the solutions found by BeesCol is high. In fact, the algorithm needs more queens to converge to a satisfactory solution. The best solution is found when using around ten queens. However, the use of a higher number of queens ($nb_Q > 20$) does not improve the solution, but increases considerably the execution time (Figure 6). Hence, it is important to use a number of queens close to 10 to get good results within a reasonable amount of CPU time.

Another parameter influencing the quality of the solution is the spermatheca size ($sper$). Indeed, one can improve the quality of solutions by choosing an appropriate spermatheca size which corresponds to number of selected drones. In Figure 5, we present the influence of the spermatheca size on the quality of the obtained solution. For each value of $Sper$, we varied the number of queens ($nb_Q \in \{5, 10, 15, 20, 25, 30\}$). The cost in Figure 5 is that of the best solution obtained for the different values of nb_Q (the other parameters being fixed). These tests are also performed on the random graph 300_5.

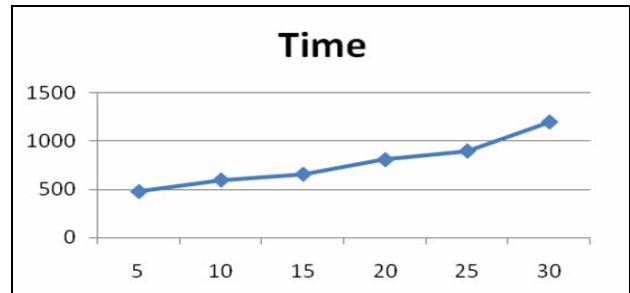
Figure 5 Influence of spermatheca size on the cost (see online version for colours)



Notes: $nb_mf = 10$; $S_0 = 0.7$; $E_0 = 0.6$; $\alpha = 0.5$ and $Q_0 = 0.8$, work = TS

From this figure, we note that use of a small spermatheca ($Sper < 5$) does not give good results. The haploid nature of the drones involves a loss of solutions diversification. Indeed, a large spermatheca ($Sper > 15$) leads to an important loss of resources, thus, the quality of the colouring achieved decreases significantly. BeesCol is able to find better colourings by using a spermatheca size between 7 and 12 ($7 \leq sper \leq 12$). This is interesting since the algorithm converges slowly when using a high number of drones.

Figure 6 Influence of nb_Q on the execution time (see online version for colours)



Notes: $Sper = 9$; $nb_mf = 10$; $S_0 = 0.7$; $E_0 = 0.6$; $\alpha = 0.5$ and $Q_0 = 0.8$, work = TS

When referring to Figure 6, it is easy to notice that the execution time is proportional to the number of queens used in the search. Using more than 20 queens, the execution time increases rapidly without improving appreciably the solution. The execution times in fig.6 are expressed in seconds. This time covers only the effective treatment

algorithms and excludes display input and output operations.

6.2 Influence of the workers on the behaviour of BeesCol

In Table 2, we present some results obtained by BeesCol when changing the worker (TS, LS, IACSCol). These tests are done respectively on the following benchmarks: 100_4, 100_5, 300_4 and 300_5. The parameters of BeesCol are those deduced from Table 1. We choose for each worker the following parameters as they give relatively the best results:

- TS: $Nb_Iteration = 5$; taboo list size = 8; neighbourhood selection mode = *Best_Move*. Recall that the neighbourhood strategy consists on moving from a partition S_i to S_j , the vertex in conflict with most of vertices in its partition S_i .
- IACSCol: $\tau = 0,01$; $\rho = 0,1$; $\alpha = 4$; $\beta = 2$, $q_0 = 0,8$; number of ants = 15; $nit = 25$; number of generation = 11; and finally for LS: the stopping criterion which consists of $Smax$ number of iterations = 7.

From Table 2, we notice that the results obtained by BeesCol for all benchmarks and workers are close to the optimum (best known results) but with different execution times. Execution times are written in the format ‘seconds: thousand seconds’. The use of TS delivers most of the time faster results. In terms of quality of the solutions, TS is the most efficient worker. As for LS and IACSCol, the results are comparable in most cases. However, LS is faster.

Table 2 Testing different workers for BeesCol

100_4 ($\chi(G) = 14$)*			100_5 ($\chi(G) = 16$)*		
Work	Cost	Time	Work	Cost	Time
IACSCol	14.53	5:432	IACSCol	16.85	21:502
LS	14.15	1:525	LS	17.13	20:384
TS	13.79	1:873	TS	15.87	19:467
300_4 ($\chi(G) = 28$)*			300_5 ($\chi(G) = 35$)*		
Work	Cost	Time	Work	Cost	Time
IACSCol	33.23	6:277	IACSCol	37.21	19:364
LS	32.92	5:872	LS	38.84	8:992
TS	29.25	4:103	TS	36.87	9:061

Note: $\chi(G)$ represents the chromatic number

Source: Costa and Hertz (1997)

Table 3 The use of BeesCol by the workers

	TS	LS	IACSCol	BeesCol with TS	BeesCol with LS	BeesCol with IACS
300-4	34.84	35.32	34.67	29.39	30.92	33.34
300-5	37.18	40.02	38.75	36.91	38.36	38.26
300-6	44.86	46.50	45.53	42.58	44.23	45.66

Table 3 shows the results obtained by TS, LS or IACSCol alone, and the results obtained by BeesCol using these same methods as worker. We used the same parameters in the two cases. Tests were run on the following random graphs: 300-4, 300-5 and 300-6 with the best parameters deduced from the preceding tables.

One can notice that the use of BeesCol allows improving appreciably the obtained results. Indeed, the number of colours decreases by two on average when using BeesCol. We also note that TS is the best worker. This is probably due to the fact that TS alone is more suited for the GCP. In fact, the search strategies used in TS are more relevant to the studied problem. However, the use of IACSCol in BeesCol does not represent a great improvement in the cost function. In IACSCol search process, ants are influenced by the pheromone trails left by the preceding ones, thus, the generated solutions are likely to be similar. While, TS improves intelligently each solution separately replacing it by the best solution found in this area, thus intensifying the search in this region. Moreover, as initial solutions (queens or broods) for TS are quite different, the generated solutions are likely to be diversified.

6.3 Comparative study between optimisation methods

In order to evaluate BeesCol (TS as worker), we compare it to some best known ant system algorithms: Ant_RLF and Ant_DSATUR presented in (Costa and Hertz, 1997). Moreover, BeesCol is compared to two ant colony system algorithms: ACS-R where an ant is implemented as RLF, and ACS_D where an ant is implemented as DSATUR (Bessedik et al., 2005).

We have done tests on several benchmarks which best-known result or chromatic number (opt) is known. Table 4 gives the optimum found by the preceding methods for the six random graphs used in Sections 5.1 and 6 other benchmarks taken from <http://mat.gsia.cmu.edu/COLOURING02>. Each value in Table 4 is the best result obtained after 10 executions.

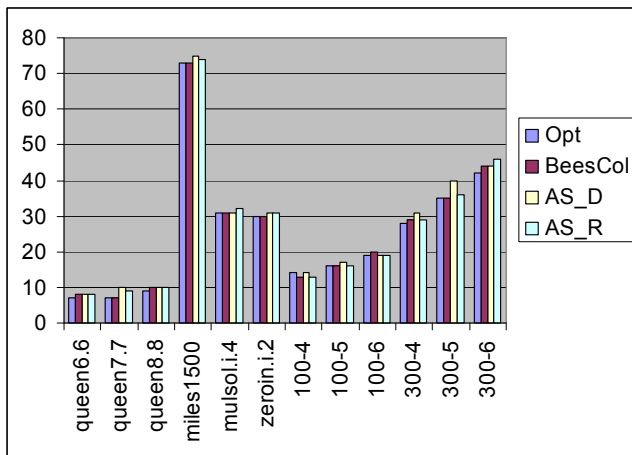
Table 4 Comparison among ants' methods

Graph	Opt	BeesCol	AS_D	AS_R	ACS_D	ACS_R
queen6.6	7	8	8	8	8	8
queen7.7	7	7	10	9	8	7
queen8.8	9	10	10	10	9	9
miles1500	73	73	75	74	73	74
multsol.i.4	31	31	31	32	32	31
zeroin.i.2	30	30	31	31	30	31
100-4	14	<u>13</u>	14	<u>13</u>	13	14
100-5	16	16	17	16	17	18
100-6	19	20	19	19	19	20
300-4	28	29	31	29	28	30
300-5	35	35	40	36	35	35
300-6	42	44	44	46	42	43

We use the following notations in Table 4: AS_D for Ant_DSATUR AS_F: for Ant_RLF ACS_D: ant colony system, construction strategy (ant implemented as DSATUR), ACS_R: ant colony system, construction strategy (ant implemented as RLF).

From Table 4 and Figure 7, we note that the preliminary results obtained by BeesCol are satisfactory compared to those of AS_D, AS_R. BeesCol reaches or gets close to the optimum in most cases except in 300_6 which chromatic number is 42. The number of colours obtained by BeesCol in this case is 44, while AS_D and AS_R give respectively 44 and 46 colours. Moreover, a close look at the benchmark 100-4 shows that BeesCol improves the best known result as do AS_R and ACS_D.

Figure 7 Comparison between BeesCol and ant system (see online version for colours)



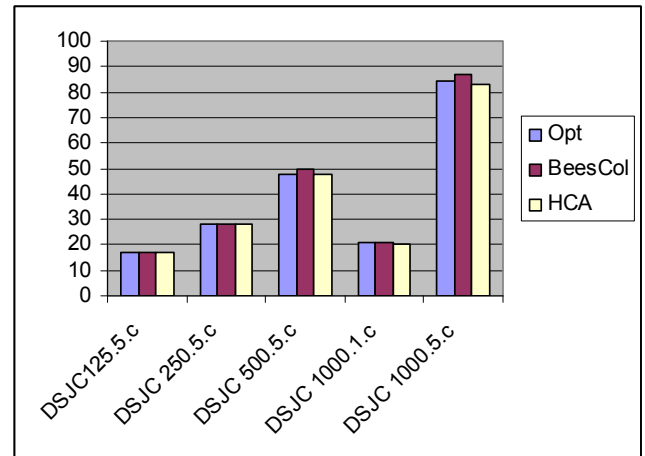
We also compared our algorithm to HCA, (Galinier and Hao, 1999) which is one of the best current algorithms for solving GCP. We made the comparison with HCA on five benchmarks used by Galinier and Hao from the well-known second DIMACS challenge: DSJ125.5.c, DSJ250.5.c, DSJ500.5.c, DSJ1000.1 and DSJ1000.5. They have 125, 250, 500 and 1000 vertices respectively and a density 0.5 except DSJ1000.1 which density is 0.1. The chromatic number of these random graphs is unknown. Each value in Table 5 is the best result obtained after seven executions.

Table 5 Comparison between BeesCol and HCA

Graph	Opt	BeesCol	ACS_D	ACS_R	HCA
DSJC125.5.c	17	17	17	18	17
DSJC 250.5.c	28	28	28	29	28
DSJC 500.5.c	48	50	49	50	48
DSJC 1000.1.c	21	21	21	22	20
DSJC 1000.5.c	84	87	89	92	83

From Table 5 and Figure 8, we notice that for DSJ125.5.c and DSJ250.5.c, both BeesCol and HCA reach the best-known result. Even though HCA performs better on the other benchmarks, BeesCol results are not far from the optimum.

Figure 8 Comparison between BeesCol and HCA (see online version for colours)



7 Conclusions and outlook for future works

In this paper, we presented an MBO approach for the GCP. In this approach, several queens are generated randomly or by a specific constructive method, RLF or DSATUR. TS or LS (workers) is used to improve each initial solution (queen) as well as each crossed one (brood). Furthermore, a proposed ant colony system algorithm (IACSCol) is considered as a worker. Since we implemented BeesCol in a sequential environment, the running time increased considerably with the spermatheca size and the number of used queens. A good compromise between the quality of the solution and the execution time is obtained by using in the search process ten queens and a spermatheca size equal to 9. The experiments showed that the use of BeesCol with TS, LS, or IACSCol as worker performed better than these methods alone. The most successful algorithm is the one using TS as worker. Moreover, it is the fastest when compared with other workers. This confirms the fact that TS is efficient when combined with evolutionary algorithms. The preliminary results obtained on some well studied DIMACS graphs show that our bee-based algorithm may be effective. Indeed, it reached most of the best known results.

BeesCol is suited for parallel computation. Therefore, the use of a parallel environment will reduce considerably the running time. The use of MBO can be suited for various other combinatory problems. We are now working on the behaviour analyse of ACS and MBO on some generalisations of the GCP such as list colouring and T-colouring.

References

- Abbass, H.A. (2001a) 'Marriage in honey bees optimization a haplometrosi polygynous swarming approach', *The IEEE Congress on Evolutionary computation (CEC 2001)*, Seoul, Korea.

- Abbass, H.A. (2001b) 'A single queen single worker honey-bees approach to 3-sat', *The Genetic and Evolutionary Computation Conference*, GECCO2001, Sans Francisco, USA.
- Abbass, H.A. (2002) 'An agent based approach to 3-sat using marriage in honey-bees optimisation', *International Journal of knowledge-based Intelligent Engineering System (KES)*, Vol. 6, No. 2, pp.1–8.
- Benatchba K. et al. (2005) 'Using bees to solve a data-mining problem expressed as a max-sat one', *International Work-Conference on the Interplay between Natural and Artificial Computation, Lecture Notes in Computer Science*, Springer-Verlag, June, Vol. 3562.
- Bessedik, M., Laib, M., Boulmerka, R. and Drias, H. (2005) 'Ant colony system for the graph coloring problem', *International Conference on Computational Modeling Control and Modeling, CIMCA 2005*, Vol. 1, pp.786–791, ISBN.
- Bonabeau, E., Dorigo, M. and Théraulaz, G. (1999) *Swarm Intelligence: From Natural to Artificial System*, Oxford University Press, New York.
- Brelaz, D. (1979) 'New methods to color vertices of a graph', *Communications of ACM*, Vol. 22, No. 251–256.
- Costa, D. (1995) 'Méthodes de résolution constructives, séquentielles et évolutives pour des problèmes d'affectation sous contraintes', These No. 1411 Lausanne, EPFL.
- Costa, D. and Hertz, A. (1997) 'Ants can color graphs', *The Journal of the Operational Research Society*, Vol. 48, No. 3, pp.295–305.
- Dorigo, M. and Gambardella, L.M. (1997) 'Ant colony system: a cooperation learning approach to the travelling salesman problem', *IEEE Trans. Evol. Comp.*, Vol. 1, No. 1, pp.53–66.
- Dorigo, M., Di Caro, G. and Gambardella, L.M. (1999) 'Ant algorithms for discrete optimization', *Artificial Life*, Vol. 5, No. 2, pp.137–172.
- Dorne, R. and Hao, J.K. (1998) 'A new genetic local search algorithm for graph coloring', *Lecture Notes in Computer Science 1498*, pp.745–745, Springer-Verlag.
- Dorne, R. and Hao, J.K. (1998) 'Tabu search for graph coloring, T-coloring an set T-coloring', S. Voss, S. Martello, I.H. Osman and C. Roucairol (Eds.): *Meta*, Chapter 6, pp.77–92.
- Fleurent, C. and Ferland, J.A. (1996) 'Genetic and hybrid algorithms for graph coloring', *Meta-heuristics in Combinatorial Optimization, Annals of Operations Research*, Vol. 63, pp.437–441.
- Galinier, P. and Hao, J.K. (1999) 'Hybrid evolutionary algorithm for graph coloring', *J. of Combinatorial Optimization*, Vol. 3, pp.379–397.
- Galinier, P. and Hertz, A. (2006) 'A survey of local search methods for graph coloring', *Computers and Operations Research*, September, Vol. 33, No. 9, pp.2547–2562.
- Gambardella, L.M. and Dorigo, M. (2000) 'An ant colony system hybridized with a new local search for the sequential ordering problem', *INFORMS Journal on Computing*, Vol. 12, No. 3, pp.237–255.
- Glass, C. and Bennett, P. (2003) 'A genetic algorithm for graph colouring: exploration of Galinier and Hao's algorithm', *Journal of Combinatorial Optimization*, Vol. 7, pp.229–236.
- Glover, F., Kelly, J.P. and Laguna, M. (1995) 'Genetic algorithms and Taboo search–hybrids for optimization', *Computers & Operations Research*, Vol. 22, p.111.
- Hamiez, J.P. and Hao, J.K. (2002) 'Scatter search For graph coloring', *Lecture Notes in Computer Science 2310*, pp.168–179, Springer Verlag.
- Hertz, A. and de Werra, D. (1987) 'Using Taboo search techniques for graph coloring', *Computing*, Vol. 39, No 4, pp.345–351.
- Koudil, M. et al. (2007) 'Using artificial bees to solve partitioning and scheduling problems in codesign', *Applied Mathematics and Computation*, 15 March, Vol. 186, No. 2, pp.1710–1722, Elsevier, doi:10.1016/j.amc.2006.08.166.
- Leighton, F.T. (1979) 'A graph coloring algorithm for large scheduling problems', *J. Res. Nat. Bur. Stand.*, Vol. 84, pp.489–506.
- Louagnouni, M. and Messaoudi, Y. (2004) 'Coloration des graphes par les algorithmes génétiques et optimisation par colonie de fourmis (ACO)', *Mémoire d'ingénierat, INI 2004*.
- Marx, D. (2004) 'Graph coloring and their applications in scheduling', *Periodica Polytechnica Ser. El.ENG*, Vol. 48, No. 1, pp.11–16.
- Murtadha Mohamad, M. (2008) 'Articulated robots motion planning using foraging ant strategy', *Journal of Information Technology – Special Issues in Artificial Intelligence*, December, Vol. 20, No. 4, pp.163–181.
- Sakarovitch, M. (1984) 'Optimisation combinatoire, programmation discrète', *HERMANN*, éditeurs des sciences et des arts.
- Smith, M.D., Ramsey N. and Holloway, G. (2004) 'A generalized algorithm for graph coloring register allocation', *SIGPLAN Not.*, Vol. 39, No. 6, pp.277–288.
- Stulzle, T. and Dorigo, M. (1999) 'ACO algorithms for the quadratic assignment problem', *Editors New Ideas in Optimization*, pp.11–32, McGraw-Hill.
- Walkowiak, K. (2005) 'Ant algorithm for flow assignment in connection oriented networks', *Int. J. App. Math. Comput. Scien.*, Vol. 15, No. 2.
- Zufferey, N. (2002) 'Heuristiques pour les problèmes de coloration des sommets d'un graphe et d'Affectaion de fréquences avec Polarité', PhD thesis, Ecole polytechnique Federale Lausanne, Switzerland.